

Model conversion

Model conversion

- 1. Model conversion
 - 1.1. CLI: pt → onnx
 - 2.2, Python: pt → onnx
- 2. Model prediction
 - CLI usage
- References

1. Model conversion

According to the test parameters of different formats provided by the Ultralytics team, we can find that the inference performance is best when using TensorRT!

when using the export mode of YOLO11 for the first time, some dependencies will be automatically installed. Just wait for it to be completed automatically!

1.1. CLI: pt → onnx

Convert the PyTorch format model to onnx and ncnn

```
cd /ultralytics/ultralytics
```

```
yolo export model=yolo11n.pt format=onnx dynamic=True #This will enable dynamic
axes to avoid static shape inference errors. For example, it supports inputs of
different batch sizes.
# yolo export model=yolo11n-seg.pt format=onnx dynamic=True
# yolo export model=yolo11n-pose.pt format=onnx dynamic=True
# yolo export model=yolo11n-cls.pt format=onnx dynamic=True
# yolo export model=yolo11n-obb.pt format=onnx dynamic=True
```

2.2, Python: pt → onnx

Convert pt model to onnx model:

```
cd /ultralytics/ultralytics/yahboom_demo
```

```
python3 model_pt_onnx.py
```

```

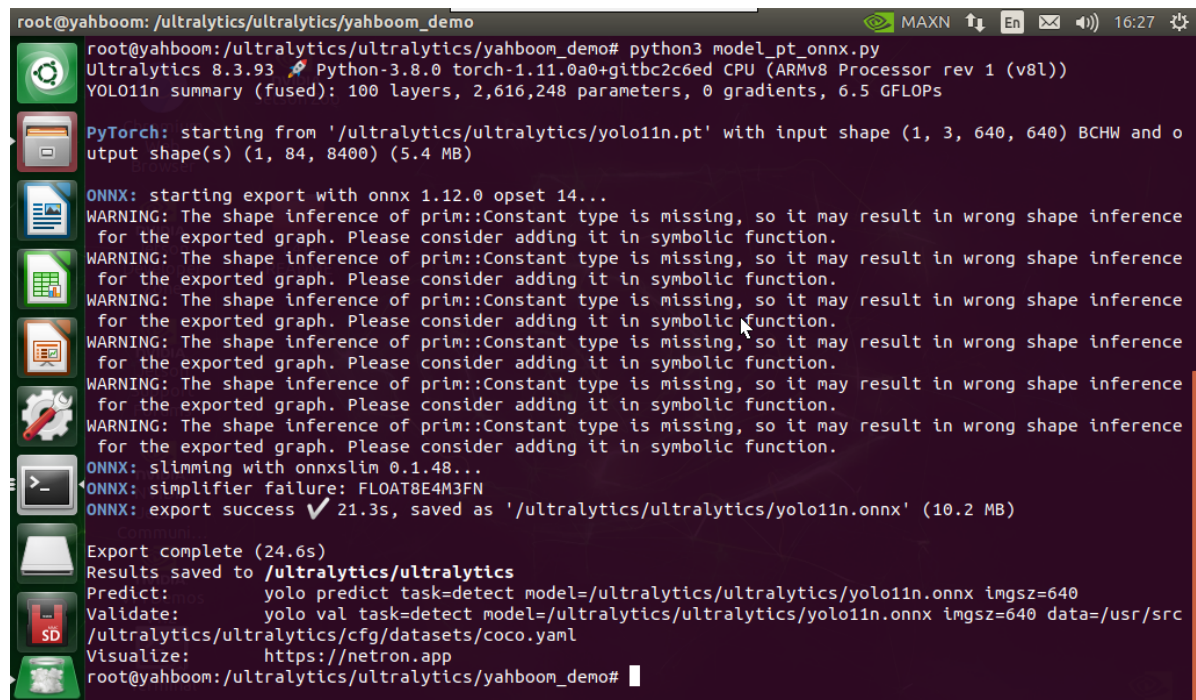
from ultralytics import YOLO

# Load a YOLO11n PyTorch model
model = YOLO("/ultralytics/ultralytics/yolo11n.pt")
# model = YOLO("/ultralytics/ultralytics/yolo11n-seg.pt")
# model = YOLO("/ultralytics/ultralytics/yolo11n-pose.pt")
# model = YOLO("/ultralytics/ultralytics/yolo11n-cls.pt")
# model = YOLO("/ultralytics/ultralytics/yolo11n-obb.pt")

# Export the model to ONNX format
model.export(format="onnx") # This will create 'yolo11n.onnx' in the same
directory

```

Note: The converted model file is located in the converted model file location



```

root@yahboom: /ultralytics/ultralytics/yahboom_demo
root@yahboom: /ultralytics/ultralytics/yahboom_demo# python3 model_pt_onnx.py
Ultralytics 8.3.93 Python-3.8.0 torch-1.11.0a0+gitbc2c6ed CPU (ARMv8 Processor rev 1 (v8l))
YOLO11n summary (fused): 100 layers, 2,616,248 parameters, 0 gradients, 6.5 GFLOPs

PyTorch: starting from '/ultralytics/ultralytics/yolo11n.pt' with input shape (1, 3, 640, 640) BCHW and o
utput shape(s) (1, 84, 8400) (5.4 MB)

ONNX: starting export with onnx 1.12.0 opset 14...
WARNING: The shape inference of prim::Constant type is missing, so it may result in wrong shape inference
for the exported graph. Please consider adding it in symbolic function.
WARNING: The shape inference of prim::Constant type is missing, so it may result in wrong shape inference
for the exported graph. Please consider adding it in symbolic function.
WARNING: The shape inference of prim::Constant type is missing, so it may result in wrong shape inference
for the exported graph. Please consider adding it in symbolic function.
WARNING: The shape inference of prim::Constant type is missing, so it may result in wrong shape inference
for the exported graph. Please consider adding it in symbolic function.
WARNING: The shape inference of prim::Constant type is missing, so it may result in wrong shape inference
for the exported graph. Please consider adding it in symbolic function.
ONNX: slimming with onnxslim 0.1.48...
ONNX: simplifier failure: FLOAT8E4M3FN
ONNX: export success ✓ 21.3s, saved as '/ultralytics/ultralytics/yolo11n.onnx' (10.2 MB)

Export complete (24.6s)
Results saved to /ultralytics/ultralytics
Predict:      yolo predict task=detect model=/ultralytics/ultralytics/yolo11n.onnx imgsz=640
Validate:     yolo val task=detect model=/ultralytics/ultralytics/yolo11n.onnx imgsz=640 data=/usr/src
/ultralytics/ultralytics/cfg/datasets/coco.yaml
Visualize:    https://netron.app
root@yahboom: /ultralytics/ultralytics/yahboom_demo#

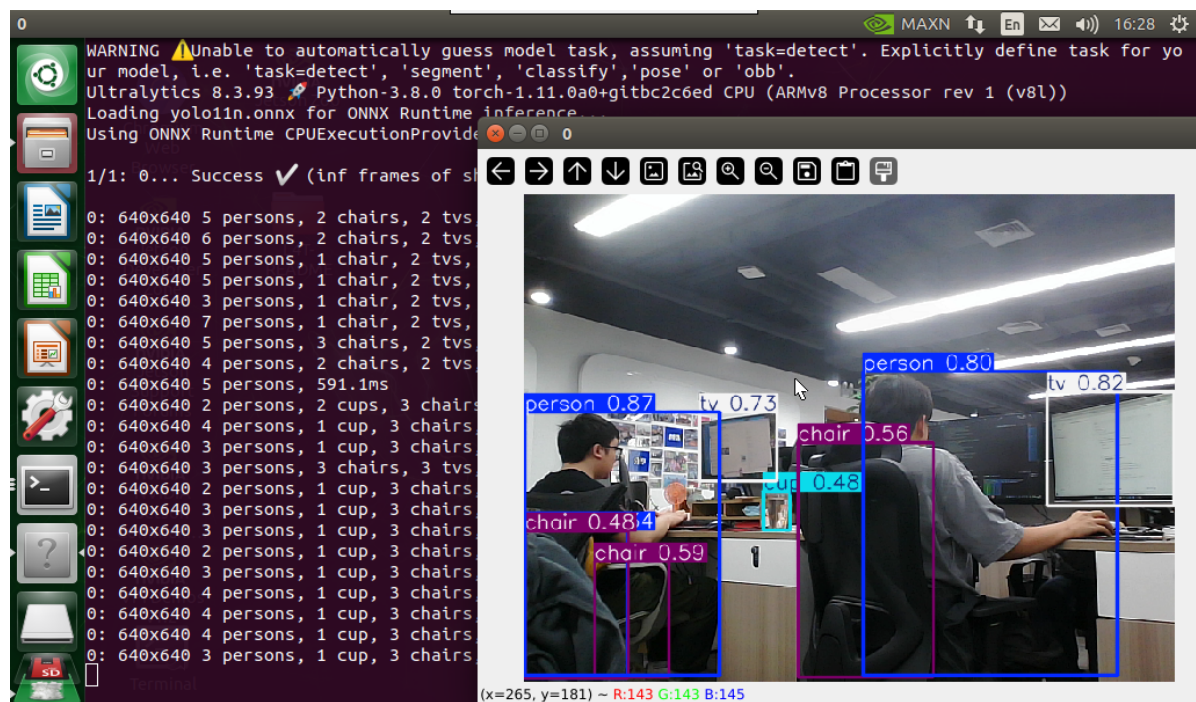
```

2. Model prediction

CLI usage

```
cd /ultralytics/ultralytics
```

```
yolo predict model=yolo11n.onnx source=0 save=False show
```



References

<https://docs.ultralytics.com/guides/nvidia-pi/>

<https://docs.ultralytics.com/integrations/tensorrt/>